

SCULPTOR Has Your Back(up Path): Carving Interdomain Routes to Services

Paper #559, 12 pages body, 19 pages total

Abstract

Cloud and content providers (Service Providers) serve diverse Internet applications — enterprise services, gaming, video, real-time AI inference — with goals that pull in different directions: low latency for some flows, high availability for others, low cost for bulk transfers, and graceful behavior when peering links fail or flash crowds arrive. To meet these goals Service Providers either We take a different approach with SCULPTOR, a framework that proactively optimizes BGP configurations to satisfy diverse ingress routing objectives-including latency, cost, and capacity constraints-simultaneously. SCULPTOR models Internet routing to approximately solve a large integer optimization problem at scale using gradient descent. We prototyped SCULPTOR on a global public cloud and tested it in real Internet conditions, demonstrating that SCULPTOR handles dynamic loads 28% larger than other solutions using existing Service Provider infrastructure, reduces overloading on links during site failures by up to 40%, and enables Service Providers to route high-priority traffic with a 50% reduction in overloading.

1 Introduction

Cloud/content providers (hereafter Service Providers) enable diverse Internet applications used daily by billions of users. Traditionally, Service Providers used DNS and static BGP advertisements to define a single path from a user to a service, leaving the specific path up to the Internet to determine.

Increasingly, however, these services have diverse requirements that can be challenging to meet with a single path that Service Providers have little control over. For example, enterprise services have tight reliability requirements [1, 2], and new applications such as virtual reality require ≤ 10 ms round trip latency [3] and ≤ 3 ms jitter [4]. Complicating matters, Service Providers must meet these requirements subject to changing conditions such as peering link/site failures [5, 6] and DDoS attacks [7, 8, 9]. Critically, such changes can cause *overload* if the Service Provider cannot handle new traffic

volumes induced by the change. This overload can lead to degraded service for users, hurting reliability [10, 11, 12, 2], and may require manual intervention.

To try to meet different goals and respond to changing conditions, Service Providers adopt solutions that are either (a) too costly, (b) too reactive, or (c) too specific. For example, Service Providers take a *costly* approach by proactively overprovisioning resources [13, 14, 15], but we demonstrate using traces that bursty traffic patterns can require overprovisioning rates as high as 70% despite sufficient global capacity (§2.3). TIPSY *reactively* responds to changing loads to retain reliability \cite{tipsy}, but reactivity could lead to short-term degraded performance. AnyOpt and PAINTER proactively set up routes to optimize *specific* objectives such as steady-state latency [16, 1], but it is challenging to extend those approaches to other objectives (§5.4.1).

We present Service Providers with a flexible framework, SCULPTOR (Scouring Configurations for Utilization-Loss-and Performance-aware Traffic Optimization & Routing), which accepts as input general objectives and outputs BGP advertisements and traffic allocations that help achieve those desired objectives. SCULPTOR is the first system that finds good solutions for ingress interdomain routing objectives such as maximum link utilization, transit cost, and latency for *inter-domain* traffic (existing systems work for intradomain traffic, e.g., [18, 19, 20, 21]). SCULPTOR computes BGP advertisements proactively without affecting live traffic, only placing live traffic on new routes after convergence, mitigating risk (§4).

To solve each traffic engineering problem, SCULPTOR efficiently searches over the large BGP advertisement search space ($> 2^{10,000}$ possibilities, i.e., from $> 10,000$ peerings) by modeling how different strategies perform, without having to predict the vast majority of actual paths taken under different configurations since predicting interdomain paths is hard and measuring them on the Internet is slow (§3.3). SCULPTOR then finds configurations using gradient descent, which is appropriate in our setting due to the horizontal scaling that gradient descent admits (§3.4). This modeling enables SCULPTOR to

assess > 1M configurations per measurement on the Internet (§5.4.1), 1,000× more than other solutions. Like other work that uses gradient descent with success (e.g., deep learning), we sacrifice the ability to provide a formal characterization of which objective functions are possible for an approach that lets us approximately optimize multiple criteria (§5)

We prototype and evaluate our framework at Internet scale using the PEERING testbed [22] (§4), which is now deployed at 32 Vultr cloud locations [23]. Vultr is a global public cloud that allows our prototype to issue BGP advertisements via more than 10,000 peerings. To demonstrate SCULPTOR’s utility across different objectives, we separately evaluate our framework on three specific objectives: (a) minimizing latency under unseen traffic conditions, (b) routing latency-sensitive and bulk traffic classes, and (c) directing traffic to lower-cost sites. We compare SCULPTOR’s performance on these problems to that of an unreasonably expensive “optimal” solution (computing the actual optimal is infeasible). We make an anonymized implementation of SCULPTOR available at https://anonymous.4open.science/r/sculptor_stripped-FE7C and will release all code upon acceptance.

For the first objective, we found that, compared to other approaches, SCULPTOR increases the amount of traffic within 10 ms of the optimal by 19.3% in steady-state (meeting that target for 95% of traffic) (§5.3.1), by 11% during link failure, and by 17% during site failure. SCULPTOR also reduces overloading on links during site failures by up to 41% over PAINTER, giving Service Providers more confidence that services will still be available during partial failure (§5.3.2). Moreover, by load balancing traffic on backup paths during peak times, SCULPTOR can satisfy high peak demands with the same infrastructure. SCULPTOR handles flash crowds (e.g., DDoS attacks) at more than 3× expected traffic volume, reducing the amount of overprovisioning that Service Providers need, reducing costs.

For the second objective, SCULPTOR routes bulk low-priority traffic in ways that avoid congesting high-priority traffic, achieving near-optimal latency and reducing congestion by up to 2× (§5.3.4).

Finally, for the third objective, SCULPTOR routes traffic with lower latency and to lower-cost sites than all other solutions (up to 3× closer to optimal) which has implications for power savings, carbon savings, and resilience (§5.3.5).

After decades of intradomain traffic engineering using programmable networking primitives such as virtual output queueing for differentiated service [24] (and others, §7), SCULPTOR brings such control closer to realization in the interdomain setting with a unilaterally deployable framework. Service Providers can use flexible frameworks such as SCULPTOR to help bring us the resilient, performant service that our diverse applications increasingly need.

Ethics. This work raises no ethical issues.

2 Motivation

2.1 Variable, Evolving Goals

Evolving Internet use cases are pushing Service Providers to meet diverse requirements. For example, Service Providers increasingly host mission-critical services such as enterprise solutions [1], which are predicted to be a \$60B industry by 2027 [25]. These enterprise solutions require high reliability, whereas low latency is a more important metric for gaming [3], which generates more revenue than the music and movie industries combined [26]. Service Providers are also expanding the set of services they provide—for example, CDNs that traditionally hosted static content are pivoting to offering services like compute [27, 28, 29].

Moreover, as use cases evolve, Service Providers increasingly need to meet performance requirements for *ingress* traffic since that traffic includes, for example, player movements in real-time games, voice and video in enterprise conferences, and video/image uploads for AI processing in the cloud [30, 31]. This reality is a departure from traditional CDN traffic patterns in which ingress traffic was primarily small requests and TCP acknowledgments.

Despite efforts to meet variable service requirements [6, 32, 33, 34, 35, 1, 15], doing so is still challenging due to issues *outside* Service Provider control.

First, Service Providers lack control over which ingress path traffic takes because the source and upstream networks determine the route. BGP computes paths in a distributed fashion where each AS selects the single best path to the destination and propagates it; Service Providers cannot force a source network to select a different path if the source prefers an alternative based on its own local preference. Service Providers cannot even unilaterally control the part of the path closest to them—upgrading peering capacity requires coordination among multiple parties [36].

Second, dynamic factors outside Service Provider control make satisfying requirements even harder. Peering disputes can lead to congestion on interdomain links and inflated paths [5, 37, 38], and DDoS attacks still bring down sites/services [9, 10, 11, 12], despite considerable effort in mitigating attacks [7, 8]. Moreover, recent work [39, 40, 21] and blog posts [41, 42, 43, 44, 45, 46] show that user traffic demands are highly variable due to flash crowds and path changes and so can be hard to plan for.

2.2 Standard Approaches

To meet evolving goals, today most Service Providers either use anycast prefix advertisements to provide low latency and high availability at the expense of some control [47, 48, 49, 50], or unicast prefix advertisements to direct users to specific sites [51, 52, 36, 53]. Anycast, where Service Providers advertise a single prefix to all peers/providers



Figure 1: In normal operation traffic is split between two sites by directing half the traffic to each prefix using DNS (a). Even though there is enough global capacity to serve all traffic when site B fails, there is no way to split traffic across multiple providers given available paths, leading to overload (b). A proactive solution is to advertise prefix 2 to an additional provider at site A *a priori*, allowing traffic splitting across the two links (c). A simpler but prohibitively expensive solution is to proactively advertise one prefix per peering (d).

at all/many sites, offers some natural availability following failures, since BGP automatically reroutes most traffic to avoid the failure after tens of seconds [49]. Prior work shows that this availability comes with higher latency in some cases [54, 48]. Unicast gives clients lower latency than anycast by advertising a unique prefix at each site [51, 53], but can suffer from reliability concerns, taking as much as an hour to shift traffic away from bad routes due to stale DNS records [1].

2.3 New Systems & Limitations

In response to some of the basic limitations of anycast and unicast, others have proposed specialized systems. However, these solutions come with three key limitations.

Too Specific: Some have noted that, by intelligently advertising different prefixes to subsets of peers/providers to offer users more paths, Service Providers can achieve better performance [5, 47, 16, 50, 1]. However, these solutions rely on objective-specific solution methods that do not easily generalize to multi-constraint optimization problems (??).

Figure 1 shows how this specific focus on a single objective could lead to reliability issues, *e.g.*, during a site failure. In normal operation, user traffic is split evenly across two prefixes and achieves low latency (Fig. 1a). However when site B fails, BGP chooses the route through Provider 1 for all traffic, causing link overutilization and subsequent poor performance for all users (Fig. 1b). In Section 5 we show that this overload happens in practice for state-of-the-art systems that provide low (steady-state) latency from users to Service Provider networks such as AnyOpt [16] and PAINTER [1].

Too Reactive: Systems like TIPSY [6] enhance reliability by withdrawing BGP announcements to drain overloaded links. However, changing the set of paths is known to lead to outages [55, 56, 57, 15], and can take time to converge. Ideally, we should modify announcements for live traffic only as a last resort.

Too Costly: Service Providers often overprovision to handle transient peak loads [13, 14, 15], but this approach can incur high costs as Figure 2 demonstrates using link utilization data (per 5 minutes) from OVH cloud [58].

To generate Figure 2 we first split the dataset into successive, non-overlapping 120-day planning periods and compute



Figure 2: Planning for peak loads to avoid overloading requires inefficient overprovisioning.

the 95th percentile ingress link utilization (“near-peak load”) for each link/period. We then assign each link’s capacity for the next period as the near-peak load in the current period multiplied by an overprovisioning factor (X-axis variable). We then compute the average link utilization in the next period and the total number of links on which we see $\geq 100\%$ utilization in at least one 5-minute interval.

Figure 2 plots the median utilization across links and periods and the number of congestive events as we vary the overprovisioning factor. Figure 2 shows that overprovisioning to accommodate peak loads introduces a tradeoff between inefficient utilization and overloading. Low overprovisioning factors between 10% and 30% lead to more efficient utilization (60%-70%) but lead to thousands of congestive events. High overprovisioning factors lead to far less overloading, but only 50% utilization. These events span 41 OVH sites and 1,650 links, indicating a widespread challenge. Results were consistent with larger planning periods, while smaller periods increased congestion.

An important note on cost.: We model deployment cost as correlated with the peering capacity over all links/sites, even though the actual relationship may be complex and additional peering capacity itself is not prohibitively expensive. Upgrading capacity has fixed costs (*e.g.*, fiber, router backplane bandwidth, line card upgrades, CDN servers near peering routers) and variable costs (*e.g.*, power, transit), both of which scale with demand [59, 14, 21, 17]. Hence the high “Link Overprovisioning Factor” to ensure low congestion should be extrapolated out to the general “cost” of upgrading Service Provider serving infrastructure, rather than just a peering link.

Efficient, Proactive Planning is Possible: Despite these limitations, Figure 1c shows that by advertising prefix 2 to provider 2 at site A *a priori*, the Service Provider can split traffic between the two links during failure, avoiding overloading without (a) reacting to the failure and (b) upgrading deployment capacity.

2.4 Key Challenges

Since a Service Provider has lots of global capacity, dynamically placing traffic on paths to satisfy performance objectives subject to capacity constraints would therefore be simple if all the paths to the Service Provider were always available to all users as in Figure 1d. However, making paths available uses IPv4 prefixes, which are expensive and contribute to the global routing table’s memory footprint. The solution in Figure 1d uses 50% more prefixes than the solution in Figure 1c and, at \$20k per prefix for 10,000 peerings would cost \$200M [60]. IPv6 is not a good alternative as it is not supported by 60% of users according to APNIC [61]. Moreover, adoption is slowing, with adoption in the US stagnating at 50% for a few years, and full adoption possibly taking until 2045 according to APNIC [62] and Google [63]. Prior work noted the same but found that advertising around 50 prefixes was acceptable [16, 1], and most Service Providers advertise fewer than 50 today according to RIPE RIS BGP data [64]. We also verified with engineers at six Service Providers that this challenge is important to address.

Since we cannot expose all the paths by advertising a unique prefix to each connected network, we must find some subset of paths to expose. Finding that right subset of paths to expose that satisfies performance objectives, however, is hard since there are exponentially many subsets to consider, and each subset currently needs to be tested (*i.e.*, advertised via BGP) to see how it performs. The number of subsets is on the order of 2 to the number of ingresses Service Providers have, which, for some Service Providers (including Vultr, which we measure) [65], is $> 2^{10,000}$. As measuring this many advertisements is intractable, we have to predict how different subsets of paths perform, which is challenging since interdomain routing is difficult to model [66, 6].

3 Methodology

3.1 Overview

SCULPTOR takes as input a Service Provider’s objective, its prefix budget, and a total training time (§3.2.2). (It also accepts required inputs to compute the objective, *e.g.*, traffic volumes and user latencies.) It outputs an advertisement configuration that gives users good interdomain paths relative to that objective, and an allocation of traffic to those paths. SCULPTOR computes BGP advertisements proactively, only placing live traffic on them after computation (§4).

To search for good advertisements and traffic allocations, SCULPTOR solves two key challenges. First, SCULPTOR does not know the objective. The only way to know exactly how users will route to a configuration is to advertise it, which can be done at most a few times per hour and so cannot be done for the millions of configurations that an optimization considers. SCULPTOR instead maintains a probabilistic (*i.e.*, noisy) model

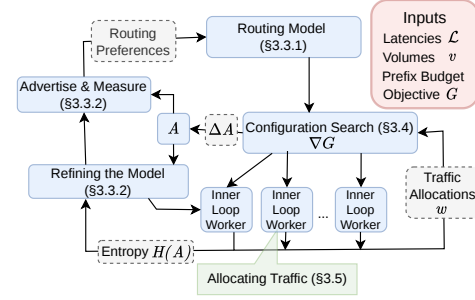


Figure 3: SCULPTOR advertisement computation overview.

of how users route, uses it to estimate the objective, and refines it with occasional measurements (§3.3).

Second, the configuration space is large and discrete. SCULPTOR searches it using gradient descent over a relaxed, real-valued configuration which helps smooth the convergence process under a noisy objective (§3.4). Each gradient entry is computed by solving a small traffic-allocation problem (§3.5).

Figure 3 shows how these pieces fit together, and in Section 5.4.1 we demonstrate that each piece is important for SCULPTOR’s success. We also outline a full iteration of SCULPTOR in Section A.2.

3.2 Problem Background

3.2.1 Setting and Routing Model

Service Providers offer their services from tens to hundreds of geo-distributed sites [67]. The sites for a particular service can serve any user, but users benefit from reaching a low-latency site for performance. Sites consist of sets of servers that have an aggregate capacity. Service Providers also connect to other networks at sites via dedicated links or shared IXP fabrics which we call peering links. Each such link also has a capacity. When utilization of a site or link nears/exceeds the capacity, performance suffers, so Service Providers strive to avoid high utilization [5, 21]. Resources can also fail due to, for example, physical failure and misconfiguration.

Users route to the deployment through the public Internet to a prefix over one of the peering links via which that prefix is advertised. The path to a prefix (therefore peering link) is chosen via BGP. We model interdomain paths from users to the Service Provider as non-overlapping except at the peering link through which each path ingresses to the deployment. We identify a path by its ingress link rather than its neighbor AS, because one AS may peer at many sites.

We assume that the peering link is the bottleneck of each path, for two reasons. First, this link increasingly represents the most important part of the path for Service Providers due to Internet flattening [68]. Second, other parts of the path are less interesting for traffic engineering—last-mile bottlenecks are common to all paths for a user, and existing systems perform traffic engineering in intermediate networks. Han-

dling unexpected bottlenecks (*i.e.*, in a peering network) can also be viewed as a path failure, which we evaluate in [Section 5.3.2](#), demonstrating that *SCULPTOR* handles violations of this assumption.

We assume the Service Provider has some way to direct each UG’s traffic among its prefixes, such as DNS [51, 53, 47, 52], multipath transport [69, 70, 71], or control points near users [72, 1]. These options differ in how deployable and how precise they are as *PAINTER* discusses [1]. *SCULPTOR* makes no particular assumption about which is used.

3.2.2 Inputs

The high-level input is the Service Provider’s *goal* which they express via an objective function, G . The Service Provider hopes to accomplish this goal via a *prefix budget* which is generally much less than the number of total ingresses, and a maximum *training time* or, equivalently, a maximum number of BGP measurements, since *SCULPTOR* is bottlenecked by BGP measurement time (§3.3).

Oftentimes, objectives are functions of how “well” users route to the deployment, so *SCULPTOR* also accepts information about users. We aggregate users into user groups (UGs), for example /24s, metro-AS pairs, or ASes. The granularity is chosen to be one at which all users in a UG route to the Service Provider the same way, so that a UG has one route per prefix (§3.2.3). We use /24s (§4).

UGs generate steady-state traffic volumes, $v(\text{UG})$, and the Service Provider provisions capacity at links/sites to accommodate this load. A system run by the Service Provider measures latency $\mathcal{L}$ and traffic volumes from UGs to Service Provider peering links l with known capacities $c(l)$, which are also known inputs in prior work [76, 53, 77, 78, 16, 1]. Part of these latency measurements includes measuring which peering links are *reachable* (in a policy-compliant routing sense) from each UG. *SCULPTOR* is resilient to changes in latency and traffic over time (§5.3.2).

3.3 The Key Goal

Finding advertisement configurations that admit good assignments of user traffic to paths is a multivariate optimization problem over both configurations and traffic assignments. [Section A.1](#) tabulates variables and expressions.

We represent an advertisement configuration A as a binary vector. Each entry indicates whether we advertise/do not advertise a particular prefix to a particular peer/provider at a particular site (similar to prior work [16, 1]). Implementing this configuration (*i.e.*, advertising prefixes via BGP sessions) results in routes from UGs to the Service Provider. A UG has one route per prefix but a UG may have their traffic fractionally assigned to different prefixes. The assignment of traffic to resulting routes from this configuration is given by the non-negative real-valued vector w , whose entries specify traffic

allocation for each UG along each route (equivalently, prefix).

Announcing a configuration will result in some set of routes (although knowing exactly which routes a priori is challenging), and these routes define which ingress link a UG uses for a prefix. We can think of this process as a routing function R that takes an advertisement configuration A and outputs a map from $\langle \text{prefix}, \text{UG} \rangle$ pairs to ingress links. For example, say $R(A) = f_A$, and $f_A(p, \text{UG}) = l$ — this notation means that the output of an advertisement configuration A under routing R is a function f_A that tells us that users UG reach prefix p via link l . It could be that a configuration leads to no route for some UG to some prefix. We define a function e such that $e(R(A)(p, \text{UG})) = 1$ when there is some route for UG to prefix p under configuration A , and 0 otherwise.

Now suppose the overall metric we want to minimize is G which is a function of both configurations and traffic assignments. Examples include traffic cost, average latency, maximum latency, and their combinations (§3.4). The joint minimization over configurations and traffic assignments can then be expressed as the following.

$$\begin{aligned} \min_{A, w} \quad & G(R(A), w) \\ \text{s.t.} \quad & w(p, \text{UG}) \geq 0; \quad A(p, l) \in \{0, 1\} \quad \forall \text{UG}, p, l \\ & \sum_p w(p, \text{UG}) e(R(A)(p, \text{UG})) = v(\text{UG}) \quad \forall \text{UG} \end{aligned} \quad (1)$$

The first constraint requires that traffic assignments be non-negative and that configurations are binary. The second constraint requires that all user traffic $v(\text{UG})$ is assigned, and none is assigned to nonexistent paths. Capacity constraints depend on the choice of G .

Solving Equation (1) is challenging because we need to compute $G(R(A), w)$, but measuring $R(A)$ exactly requires advertising prefixes in the Internet, which can only be done infrequently to avoid route-flap-dampening and allow convergence. The optimization problem requires evaluating $G(R(A), w)$ for millions of different A , which could take a hundred years at a rate of advertising one configuration every 20 minutes.

Moreover, even if R were known, Equation (1) is not a tractable optimization problem since (1) G depends on A only through R , which is the Internet’s routing rather than an algebraic function of A , and (2) the resulting MIP problem is too large for off-the-shelf solvers.

$$\min_A \quad \mathbb{E}_R \left[\min_w G(R(A), w) \right] \quad \text{s.t. constraints of Equation (1)} \quad (2)$$

In the remainder of this section we describe each of the three components in Equation (2). The expectation over R represents what we know about routing and how we learn more (§3.3). The outer minimization over A is how we move through configuration space (§3.4.2). The inner minimization over w is how we allocate traffic once routes are fixed (§3.5),



Figure 4: SCULPTOR initially estimates latency from this UG to both the red and blue prefixes with the average over possible ingress latencies (4a). A priori, no prefix is advertised. SCULPTOR then advertises the red prefix and learns that ingress I is preferred over III and IV (4b). SCULPTOR uses this information to refine its latency estimate towards the blue prefix (ingress III is not a preferred option) without advertising the blue prefix, saving time.

and it is where a Service Provider’s specific objective, G , plugs in (§5.1.2).

3.4 Modeling & Measuring Routes

In Equation (2) we compute expectations over a random process that represents the routing, R , since we cannot exhaustively measure R . Specifically, SCULPTOR models UG paths probabilistically and improves its model over time through relatively few measurements.

3.4.1 Routing Model

A priori, for a given UG towards a given prefix, SCULPTOR treats all reachable ingress link options to which that prefix is advertised as equally likely. One could incorporate prior information about ingress preferences (from economic/routing models, for example), but such complexity is unnecessary for good performance.

We instead learn preferences over time via BGP measurements and observe which ingress the UG takes (§3.3.2). Upon learning that one ingress is preferred over the others receiving that prefix advertisement, we exclude that less-preferred ingress as an option for that UG in all future calculations for all prefixes for which both ingresses are an option. Prior work used a similar routing model [16, 66, 1] but did not extend it to deal with general objectives nor did it treat routing probabilistically.

As we learn which paths UGs prefer the most, our estimate of G on unmeasured configurations concentrates on the true value, allowing us to more efficiently carry out Equation (2). An example of this learning process is shown in Figure 4—SCULPTOR refines a latency estimate towards an unmeasured prefix (blue) using measurements towards other prefixes (red). Even though SCULPTOR has 50% confidence in which path the user takes (*i.e.*, in $R(A)$), SCULPTOR knows that the user’s latency towards the blue prefix will be about 23 ms (*i.e.*, either 26 ms or 20 ms) and so SCULPTOR obtains a better estimate of the objective.

3.4.2 Refining the Model

Probabilistic estimates of G could be inaccurate until we learn UG preferences. To refine its model, SCULPTOR periodically measures $R(A)$ on its current configuration (§3.4) by advertising prefixes and measuring UG preferences by observing which routes users take. To use its BGP measurement budget wisely, SCULPTOR makes two design decisions:

What to measure: SCULPTOR measures the *current* configuration (as opposed to random or slightly different ones) since it uses a descent-based optimization (§3.4). This type of optimization compares the current advertisement to many adjacent/different ones, so knowing the current value is much more important than knowing adjacent ones.

When to measure: SCULPTOR measures either when (a) its uncertainty (according to Monte Carlo simulations, §3.5) exceeds a certain threshold or (b) SCULPTOR has not executed a measurement in many learning iterations. There are many possible choices of “uncertainty”, we choose entropy [79]. Intelligently choosing when to measure offers a slight performance improvement over a fixed measurement schedule (§5.4.1).

3.5 Searching Over Configurations

We now describe the outer optimization loop in Equation (2) over advertisement configurations, A .

3.5.1 Initializing A

We initialize our configuration to be *anycast* on one prefix and *unicast* on remaining prefixes (*i.e.*, one prefix per site), since both are well-understood configurations with complementary strengths (§2.2). Remaining prefixes in the budget (if they exist) are advertised to random subsets of ingresses.

3.5.2 Gradient Descent

For the reasons discussed in Section 3.2.3 we cannot find A simply using an off-the-shelf solver. A reasonable choice to solve such problems are descent-based algorithms—ones that compare adjacent options to the current best guess and pick an adjacent option as the next configuration. However, an *additional* challenge in our setting is that we only have noisy estimates of the objective, so placing too much trust in objective estimates of adjacent configurations can lead to poor convergence.

SCULPTOR instead uses *gradient* descent since gradient descent slowly incorporates and remembers information about how useful each advertisement is rather than overcommitting, which is what greedy and random searches used by prior systems do [16, 1].

To apply gradient descent to our problem, we create a real-valued auxiliary variable to A , $\tilde{A}$, with entries between 0 and 1 representing “confidence” values that an advertisement to the

corresponding ingress will be beneficial. For coordinate (p, l) , SCULPTOR evaluates the inner objective (§3.5) with $A(p, l)$ set to 1 and then to 0, holding the rest of the thresholded configuration fixed. The difference between these two values, scaled by the derivative of a sigmoid at $\tilde{A}$, is the gradient entry (similar to prior work [80]). Since there are too many gradients to compute, we subsample gradient entries and track the largest ones (as in prior work [81]). We then threshold $\tilde{A}$ at 0.5 to translate real-valued confidences to whether or not we should advertise a prefix at an ingress (*i.e.*, thresholded $\tilde{A} = A$).

We implement SCULPTOR with Nesterov’s Accelerated Gradient Descent (§3.4.2) in Python [82]. We use RMSProp with an initial learning rate of 1.0.

Scaling: Gradient descent scales with the product of the number of ingresses and number of prefixes but individual entry computation is independent and therefore parallelizable.

Convergence: Gradient descent converges to a local minimum for bounded objectives [83]. Our evaluations show that SCULPTOR finds good solutions over a wide range of emulated topologies (§4.1) and converges quickly with thousands of UGs and $\langle \text{peering}, \text{prefix} \rangle$ pairs (§5). Allocating higher prefix budgets and adding richer advertisement capabilities (*e.g.*, BGP community tagging) can lead to convergence to a better minimum, which is an area for future work.

3.5.3 Termination

SCULPTOR stops gradient descent either when the BGP measurement budget (*i.e.*, time budget) has been exhausted or the average change in objective over iterations falls below a small threshold. We stop training when measurement budget has been exhausted since SCULPTOR may make poor decisions if it has a bad estimate of the routing function.

3.6 Allocating Traffic

This section specifies the inner minimization over traffic allocations in Equation (2).

Given our routing model (§3.3.1) and current advertisement (§3.4), we Monte Carlo sample routing functions R . Each sample independently draws, for every UG and prefix, one ingress uniformly from the ingresses the model still considers possible for that pair (§3.3.1). For each sample we then solve the traffic allocation exactly with a general-purpose solver, and the expectation in Equation (2) is the average of those solutions. In practice we find that a few Monte Carlo samples per gradient entry is sufficient.

Given a sampled routing realization, solving for the optimal traffic allocation is then a problem in classical (*i.e.*, intradomain) traffic engineering. We solve traffic allocations (§3.5) with SciPy HIGHS [84].

Scaling: This component scales with the number of Monte

Carlo samples and the scaling behavior of each traffic allocation, which depends on the objective, G .

Convergence: depends on the objective, G . SCULPTOR accepts any G for which, given fixed routes, the allocation problem is efficiently solvable. All objectives that we evaluate in Section 5.1.2 are linear programs.

4 Evaluation Setup

We prototype SCULPTOR on the PEERING testbed [22], which is now available at 32 Vultr cloud sites. We describe how we built SCULPTOR on the real Internet and how we *emulate* a Service Provider including their clients, traffic volumes, and resource capacities. (We are not a Service Provider and so could not obtain actual volumes/capacities, but our extensive evaluations (§5) demonstrate SCULPTOR’s potential in an actual Service Provider and our open/reproducible methodology provides value to the community.)

In practice, Service Providers can use SCULPTOR *safely* by only shifting production traffic onto prefixes after SCULPTOR’s announcement exploration has converged and by integrating SCULPTOR into automated route configuration frameworks. More details about how a Service Provider may do so are in Appendix B.

4.1 Emulating Client Traffic

We measure actual client latency from IP addresses to our PEERING prototype as in prior work [1], selecting targets according to assumptions about Vultr cloud’s client base.

We first tabulate a list of 5M IPv4 targets that respond to ping via probing each /24. Vultr informs cloud customers of which prefixes are reachable via which peers, and we use this information to tabulate a list of peers and clients reachable through those peers. We then measure latency from all clients to each peer individually by advertising a prefix solely to that peer using Vultr’s BGP action communities and pinging clients from Vultr. We also measure performance from all clients to all providers individually, as providers provide global reachability.

In our evaluations, we limit our focus to clients who had a route through at least one of Vultr’s direct peers (we exclude route server peers [85]). Vultr likely peers with networks with which it exchanges a significant amount of traffic [36], so clients with routes through those peers are more likely to be “important” to Vultr. We found 700k /24s with routes through 1086 of Vultr’s direct ingresses. We removed clients whose lowest latency to Vultr was 1 ms or less, as these were assumed to be addresses related to infrastructure, leaving us with measurements from 666k /24s to 825 Vultr ingresses.

We emulate traffic volumes in two ways: First, we randomly choose the total traffic volume of a site as a number between 1 and 10 and then divide volume up randomly among clients that anycast routes to that site. Client volumes in a site are

set to be within one order of magnitude of each other. While potentially unrealistic, testing many different random volumes demonstrates SCULPTOR’s robustness to assumptions about a particular distribution. We avoid Zipf-like distributions as they lead to trivial solutions where a small fraction of sites dominate importance.

Second, we derive realistic volumes using APNIC AS populations [86], shown to be representative of Service Provider traffic volumes [87]. We assign each AS its aggregate population and divide it evenly among its clients.

We found that the high-level results shown in Section 5 were the same using both emulated and APNIC volumes, and so report results for emulated traffic.

4.2 Deployments

Experiments in the Internet. We assess how SCULPTOR performs on the Internet using RIPE Atlas probes [88], which represent a subset of all clients. RIPE Atlas allows us to measure paths (and thus ingress links) to prefixes we announce from PEERING, which SCULPTOR needs to refine its model (§3). We limit the scale of our deployment to 10 sites to avoid reaching RIPE Atlas daily probing limits (15k traceroutes/day). We prioritize geographic density over coverage to create a challenging routing surface with subtle unicast differences. These 10 sites include major hubs across North America, Europe, and South America. Choosing RIPE Atlas probes to maximize network coverage and geographic diversity, we select probes from 972 networks in 38 countries which have paths to 484 unique ingresses. Each probe has paths via approximately 60 ingresses. We use 12 prefixes and change advertisements at most once per hour to avoid route-flap-dampening.

Emulations. We also evaluate SCULPTOR by emulating user paths which allows us to conduct more extensive evaluations, as experiments take less time and use clients in more networks. We compute solutions over many random routing preferences, demands, and subsets of sites to demonstrate that SCULPTOR’s benefits are not limited to a specific deployment. We evaluate SCULPTOR over deployments of size 3, 5, 10, 15, 20, 25, and 32 sites. Each size deployment is run at least 10 times with random subsets of UGs, UG demands, and routing preferences. The distribution of the number of /24s per peer is Pareto-like, so we consider random subsets of /24s through each ingress in a way that balances the number of unique /24s per ingress. Over all scenarios, we consider paths from clients in 52k prefixes (representing 31% of APNIC population [86]) to 873 ingresses. We use one tenth of the number of ingresses as the number of prefixes in our budget (10 prefixes for 3 site deployments, 60 prefixes for 32 site deployments). Prior work found that using tens of prefixes to improve performance was a reasonable cost [16, 1].

4.3 Setting Resource Capacities

We assume that resource capacities are overprovisioned proportional to their usual load. However, we do not know the usual load of Vultr links and cannot even determine which peering link that traffic to one of our prefixes arrives on, as Vultr does not give us this information. (This limitation exists since we are not a Service Provider, but a Service Provider could measure this using IPFIX, for example.) We overcome this limitation using two methods corresponding to our two deployments in Section 4.2.

Experiments in the Internet. For our first method of inferring client ingress links, we advertise prefixes into the Internet using the PEERING testbed [22], and measure actual ingress links to those prefixes using traceroutes from RIPE Atlas probes [88]. Specifically, we perform IP to AS mappings and identify the previous AS in the path to Vultr. This approach has limited evaluation coverage, as RIPE Atlas probes are in a few thousand networks. In cases where we cannot infer the ingress link from a traceroute, we use the closest latency from the traceroute to the clients’ (known) possible ingresses. For example, if an uninformative traceroute’s latency was 40 ms to Vultr’s Atlanta site and a client was known to have a 40 ms path through AS1299 at that site, we say the ingress link was AS1299 at Atlanta.

Emulations. The second method we use to infer ingress links is emulating user paths by assuming we know all user routing preferences (§3.2). We use a preference model where clients prefer peers over providers, and clients have a preferred provider. When choosing among multiple ingresses for the same peer/provider, clients prefer the lowest-latency option. We also add in random violations of the model. This approach allows us to evaluate our model on *all* client networks but may not represent actual routing conditions, though prior work found it held in 90% of the cases they studied. We conducted a sensitivity analysis on this model/assumption and found that SCULPTOR’s performance gains are robust to variations (*e.g.*, random preferences).

Given either method of inferring client ingress links (RIPE Atlas/emulations), we then measure paths to an *anycast* prefix and assign resource capacities as some overprovisioned percentage of this catchment. (Discussions with operators from Service Providers suggested that they overprovision using this principle.) We report results for an overprovisioning rate of 10%, but find similar takeaways for 10% through 50%.

5 Evaluation

5.1 Evaluation Setting

5.1.1 Baseline Solutions

anycast.: A single prefix announcement to all peers/providers at all sites, which is a common configuration used by Service Providers today [89, 90, 47, 91, 92, 93, 50].

unicast.: A single prefix announcement to all peers/providers at each site (one per site). Another common configuration used in today’s deployments [51, 52, 53].

AnyOpt & PAINTER.: Two proposed strategies for reducing steady-state latency compared to anycast [16, 1]. AnyOpt was one of the first works to systematically optimize ingress traffic, while PAINTER built on that effort. (We do not compute the solution for AnyOpt for our evaluations on the Internet since AnyOpt did not perform well compared to any other solution in our emulations, and since AnyOpt takes a long time to compute.)

One-per-Peering.: A unique prefix advertisement to each peer/provider, so many possible paths are always available from users. This solution serves as our performance upper-bound, even though it is prohibitively expensive. (We do not know an optimal solution with fewer prefixes.)

We compute both average overall latency and the fraction of traffic within 10 ms (very little routing inefficiency), 50 ms (some routing inefficiency), and 100 ms (lots of routing inefficiency) of the One-per-Peering solution for each advertisement configuration, as these statistics provide a more informative measure of latency improvement than averages.

5.1.2 Objectives Evaluated

While our approach likely accommodates other objectives [17, 36, 94, 1, 16, 53, 21, 18, 19, 95], it provides practical utility at the expense of a formal characterization of feasible functions, similar to prior gradient descent work. We write down the objective for (1) as an example (our other objectives often have similar structure).

Optimizing For Latency and MLU. Under our model (§3.2.1), steady-state path latency for a UG to a prefix is uniquely determined by UG and the peering link over which the UG ingresses (but latency may change over time, which we evaluate in Section 5.3.2). Let the latency for a user UG via a link l be $\mathcal{L}(UG, l)$. G is then given by the following.

$$G(R(A), w) = \frac{1}{\sum_{UG} v(UG)} \sum_{p, UG} \mathcal{L}(UG, R(A)(p, UG)) w(p, UG) + \beta M$$

$$\text{s.t. } \frac{\sum_{R(A)(p, UG)=l} w(p, UG)}{c(l)} \leq M \quad \forall l \quad (3)$$

For link capacity $c(l)$ minimizing Equation (3) forces M to be the maximum link utilization (MLU). β weights a tradeoff between using uncongested links/sites and low propagation delay and is set by the Service Provider based on their goals. We first solve with $M = 1$ to see if we can allocate traffic to paths with zero overloading, which may not be possible for all A .

Regularizing For Failure Robustness. To encourage robustness under unseen conditions (*e.g.*, failures), we add $\sum_l \alpha_l G(R(A * F_l), w)$ to G . Binary vectors F_l are defined so



Figure 5: SCULPTOR outperforms other solutions on the Internet deployment, reducing steady-state latency (a) and congestion during link failures (b).

that multiplying advertisements, A , by these vectors models withdrawal/failure on a link/site (*i.e.*, zeroing out the corresponding components). This regularizer encourages solutions with good primary and backup paths. We choose $\alpha_l = 0.1$ in our evaluations.

Enabling Latency-Sensitive Services. Some services have very tight latency requirements (*e.g.*, ≤ 10 ms) [3]. This objective aims to maximize the amount of traffic within 10ms of its One-per-Peering latency. (We choose the difference with One-per-Peering latency to isolate SCULPTOR’s goal of minimizing routing inefficiency.)

Supporting Multiple Traffic Classes. Similar to private WAN traffic engineering [19, 18], we consider routing traffic with distinct performance requirements. We consider a scenario where one traffic class (high-priority) should be routed with low latency, and the other (low-priority) should not congest the high-priority traffic. An additional challenge in the interdomain setting is that we cannot use priority queueing to ensure that high-priority traffic is not congested since we do not control queue behavior on the path. The objective is a weighted combination of the average latency of high-priority traffic and the amount of high-priority traffic that is congested. We also set a limit on the maximum overload on any link so that not all low-priority traffic lands on one link, as this solution would lead to poor low-priority goodput.

Minimizing Traffic Cost Across Sites. We also consider the scenario where each site has a cost proportional to the traffic volume arriving at the site. The cost per unit can vary across sites. This objective is a simple model of scenarios where Service Providers want to minimize power/carbon/peering infrastructure cost (which can vary by region) or minimize the use of unreliable sites (*e.g.*, dirty power). We minimize the weighted sum of average latency and average site cost. Our goal is to demonstrate that our approach supports trading off across these types of goals, not to argue that our simple cost model is correct.

5.2 On-Internet Experiments

We evaluate the (a) MLU and (b) failure robustness objectives on our Internet deployment (§4.2).

5.2.1 Steady-State Latency and MLU

Figure 5a shows a CDF of the difference in latency between each solution and One-per-Peering for all UGs, weighted by traffic. SCULPTOR outperforms other solutions and is only 2.0 ms worse than (the unreasonably expensive) One-per-Peering solution on average, whereas the next best solution, PAINTER, is 5.5 ms worse. SCULPTOR also serves 91.8% of traffic within 10 ms of the latency with which One-per-Peering serves it, whereas the same is true for only 88% of traffic for PAINTER.

5.2.2 Failure Robustness

Figure 5b demonstrates that SCULPTOR offers lower latency paths for more UGs during single link failure in realistic routing conditions. On average, SCULPTOR is 7.9 ms higher latency than One-per-Peering, compared to unicast which is 14.2 ms higher than One-per-Peering. PAINTER struggles to find sufficient capacity for UGs, overloading 69.6% of traffic.

Site failures show similar results. On average, SCULPTOR is 13.1 ms higher latency than One-per-Peering, while the next-best solution, unicast, is 25.1 ms higher. PAINTER again performs poorly, with 95% of traffic overloaded during site failure.

5.3 Emulated Experiments

We extensively evaluate SCULPTOR against other solutions across all objectives in our emulated deployments. Table 1 demonstrates that SCULPTOR outperforms all other solutions on deployments. Each objective was trained and computed on **TBD: 10** emulated deployments.

For context, at the scale of Service Providers that serve trillions of requests per day, improving a few percent of traffic by tens of milliseconds represents a significant improvement. Service Providers recently emphasized that small percentage gains are important [96, 97].

5.3.1 Steady-State Latency and MLU

Average latency for SCULPTOR is on average 1.3 ms worse than One-per-Peering while the next-best solution (PAINTER) is on average 3.4 ms worse than SCULPTOR. Interestingly, unicast (5.1 ms worse than One-per-Peering) performs better than AnyOpt (17.1 ms worse than One-per-Peering) which may be because AnyOpt was designed to minimize latency without capacity constraints while unicast naturally provides many high-capacity paths to users. SCULPTOR obtains the same MLU as One-per-Peering in all cases.

5.3.2 Failure Robustness

We next assess SCULPTOR’s ability to robustly handle failures. Failures here could represent excessive DDoS traffic on the link/site (thus using the link/site as a sink for the attack), physical failure, resource draining, changes in latency on a path, congestion in the public Internet, and planned maintenance. Table 1 demonstrates that SCULPTOR shifts traffic without overloading alternate links/sites more effectively than other solutions, *without reactive BGP changes* which could cause further failure (§2.3).

In solving for traffic allocations during failure scenarios, links may be overloaded. We say all traffic arriving on a congested link is congested. We separately note the fraction of traffic that lands on congested links and do not include it in average latency computations, but say such traffic does *not* satisfy 10 ms, 50 ms, or 100 ms objectives.

Performance Under Ingress/Site Failure We fail single links and find that and find that most solutions have zero congestion since, with 32 sites, there are many backup options. SCULPTOR, however, is only 6 ms worse than One-per-Peering on average during these failures while unicast (the next-best solution) is 17 ms worse on average. During more extreme site failures, SCULPTOR only congests 2.22% of traffic on average whereas PAINTER congests 14% of traffic.

Efficient Infrastructure Utilization In training for failure robustness, we find that SCULPTOR enables distributing higher-than-normal load over existing infrastructure. We quantify this improved infrastructure utilization under two realistic traffic patterns — flash crowds and diurnal patterns.

Methodology. We define a flash crowd as traffic increase for users in a region. Examples include content releases that spur downloads in a particular region and localized DDoS attacks. Since increased demand is localized, we can spread excess demand to other sites, which is a cheaper option than installing more capacity (see Section 3.2.1 for our cost model). Links are still provisioned 10% higher than anycast traffic volume as in the rest of Section 5.3.2.

We identify each client with a single “region” corresponding to the site at which they have the lowest possible latency ingress link. For each region individually, we then scale each client’s traffic in that region by $M\%$ and compute traffic to path allocations. If there are S sites in the deployment, we thus compute S separate allocations per M value where each allocation assumes inflated traffic in exactly one region, but all the allocations are across a single set of routes calculated based on the original (non-flash) traffic. For a target region, we increase M until any link experiences overloading, then find the lowest such M value across regions. For example, if a 60% increase in traffic for Atlanta clients creates overload

Method	Latency + MLU		Failure Robustness					Traffic Classes		Latency Sensitive Services	Traffic Cost Across Sites
	Latency (ms)	MLU	Latency (ms)	% cong ingress-fail	% cong site-fail	Flash crowd intensity	Diurnal intensity	HPrio latency (ms)	Crit bulk ratio	% within 10ms of optimal	Wgt avg site cost
One-per-Peering	29.05	0.88	29.34	0.00	0.20	34.40	14.10	29.04	5.62	87.12	0.5436
SCULPTOR	30.39	0.88	30.31	0.00	2.22	33.64	13.07	31.34	4.35	83.70	0.5574
PAINTER	33.71	0.90	33.44	0.00	14.05	30.97	10.62	33.75	1.29	79.00	0.5802
unicast	34.17	0.89	34.64	0.00	15.02	33.64	12.66	34.24	2.74	75.97	0.5644
AnyOpt	46.19	0.91	47.29	8.88	88.45	30.21	10.21	46.14	1.19	68.20	0.5875
anycast	51.22	0.91	52.02	89.02	92.43	30.21	9.80	51.75	0.25	67.33	0.5921

Table 1: Performance of all methods across objectives. SCULPTOR outperforms every methodology on every metric.

(and no values $< 60\%$ did for any region), we call $M = 160\%$ the critical value of M .

We use a similar methodology for diurnal traffic analysis. We define a diurnal effect as a traffic pattern that changes volume according to the time of day. Diurnal effects might be different for different Service Providers, but a prior study from a Service Provider demonstrates that these effects can cause large differences between peak and mean site volume [21]. We sample diurnal patterns from that publication and apply them to our own traffic. We group sites in the same time zone and assign traffic in different time zones different multipliers — in “peak” time zones we assign a multiplier of M and in “trough” time zones a multiplier of $0.1M$. The full curve is shown in Appendix C. Similar to our flash crowd analysis, we increase M until at least one link experiences overloading at least one hour of the day.

Results. Table 1 shows the average critical M value over deployments that cause overloading under flash crowds and diurnal effects computed using emulated deployments. SCULPTOR is able to route traffic with **TBD: 33.64**× normal volume in all regions, whereas the same is true for 30.97% for PAINTER. Similarly, SCULPTOR achieves a diurnal intensity of 13.07 for diurnal traffic whereas the next-best solution is unicast with 12.66.

5.3.3 Enabling Latency-Sensitive Services

We also evaluate SCULPTOR’s ability to enable latency-sensitive services that have hard latency requirements such as VR/AR with a requirement of 10 ms [3]. Specifically, we try to maximize the amount of traffic to within 10 ms of its lowest latency option. This objective is highly non-linear (a step function) and so challenging to optimize.

Table 1 demonstrates that SCULPTOR pushes 4% more traffic globally to within this threshold than PAINTER and, compared to anycast, pushes us 83% of the way to the One-per-Peering solution.

5.3.4 Handling Multiple Traffic Classes

We also evaluate SCULPTOR’s ability to satisfy multiple traffic classes. We split traffic into high and low-priority. The objective is to route high-priority traffic to low-latency routes while limiting congestion on those routes from low-priority traffic. We do not penalize cases where low-priority is congested, but do limit the maximum amount of any traffic on a link to $10\times$ the capacity of the link to avoid solutions where all low-priority is placed on one link, as this solution would lead to low goodput for low-priority traffic (in practice, UGs would lower sending rates in response to congestion). We solve SCULPTOR on a single emulated 32 site deployment.

In Figure 6b we vary the amount of low-priority traffic as a multiple of high-priority traffic volume and compute the fraction of high-priority traffic congested. Links are provisioned to $5\times$ the high-priority traffic volume (different from Section 5.3.2), as that is roughly the $L_{\text{Prio}}/H_{\text{Prio}}$ ratio reported in prior work [19, 18]. There is insufficient global capacity to route all traffic for all $L_{\text{Prio}}/H_{\text{Prio}}$ over 4.0, thus the jump in anycast congestion in Figure 6b. SCULPTOR finds strategies that allow us to route more low-priority traffic with less congestion than all other approaches. For example, when $L_{\text{Prio}}/H_{\text{Prio}} = 5$, SCULPTOR achieves half as much congestion as PAINTER and unicast.

Figure 6a also shows that SCULPTOR routes traffic with lower latency than other solutions (intercepts at the right of the graph show fractions of high-priority traffic not congested). Figure 6a uses $L_{\text{Prio}}/H_{\text{Prio}} = 4$, a midpoint between the ratios seen in SWAN [19] and B4 [18].

5.3.5 Minimizing Per-Site Traffic Costs

We also evaluate SCULPTOR’s ability to jointly minimize both latency and per-site traffic cost, since different sites may cost more or less to serve traffic due to, for example, the availability of power. We solve SCULPTOR on three emulated 32 site deployments (§4.2), using different per-site costs: (a) completely random per-site costs, (b) a model of expensive vs cheap sites where half the sites were $1.2\times$ the cost of other sites, and (c) a model of per-site cost based on the grams of CO_2 emitted per